GEUREFLECTOR DOCUMENTATION

Mesh Topology Examples

A visual cookbook of trunk, satellite and twin layouts — European running example with Italian BrandMeister Zone prefixes

GeuReflector — an extended fork of SvxReflector. Drop-in replacement for the upstream `svxreflector` that adds trunks, twins, satellites, filters and more.

Built on the SvxLink project by SM0SVX — github.com/sm0svx/svxlink
Thanks to DG1FI, DJ1JAY, SA2BLV, UR3QJW, YO6NAM and others for their interest, testing, debugging, suggestions, and contributions.

PDF generated 2026-07-10 12:27 UTC
github.com/audric/GeuReflector

Background

GeuReflector links separate reflector instances in two distinct ways:

Trunk (TCP `:5302`) — bidirectional, designed for full-mesh peering. Each peer owns one or more numeric prefixes. TG routing is by **longest-prefix-match** against the union of all prefixes in the mesh (every `TrunkLink` learns the full table at startup). Audio relay is **single-hop**: only the prefix owner fans audio out to other interested peers. Non-owners that receive trunk audio deliver it locally and stop. The source peer is excluded from the owner's fanout to prevent loops.

Satellite (TCP `:5303`) — one-to-many star. A satellite reflector connects out to a parent reflector and mirrors TGs in both directions. An optional `SATELLITE_FILTER` on the satellite side scopes the mirrored set in **both** directions: the satellite suppresses non-matching outbound, and the parent learns the filter via `MsgPeerFilter` (type 122) and drops non-matching forwards going down.

Two other behaviours show up repeatedly in the flow descriptions below:

- **Peer interest** is populated either when a peer emits `PeerTalkerStart` / `PeerAudio` on a TG, or proactively when a peer advertises its local clients' selected / monitored TGs via `MsgPeerTgInterest` (type 129). The proactive path means a passive listener — a SvxLink node that has selected a foreign TG or added it to `monitoredTGs` without keying — registers as interested at the owner without first transmitting; the receiver re-advertises one hop further toward the owner so multi-hop national-mesh layouts (§2) propagate correctly. Interest expires after **10 minutes** of inactivity on that TG from that peer (or immediately on trunk disconnect).
- **MCC-style country prefixes** are used for the international mesh: IT = `222`, SE = `240`, DE = `262`. Reusing the DMR MCC layout makes ownership self-documenting.



1. International full-mesh

Country gateways only. Every reflector owns its country's MCC and trunks directly to every other country.

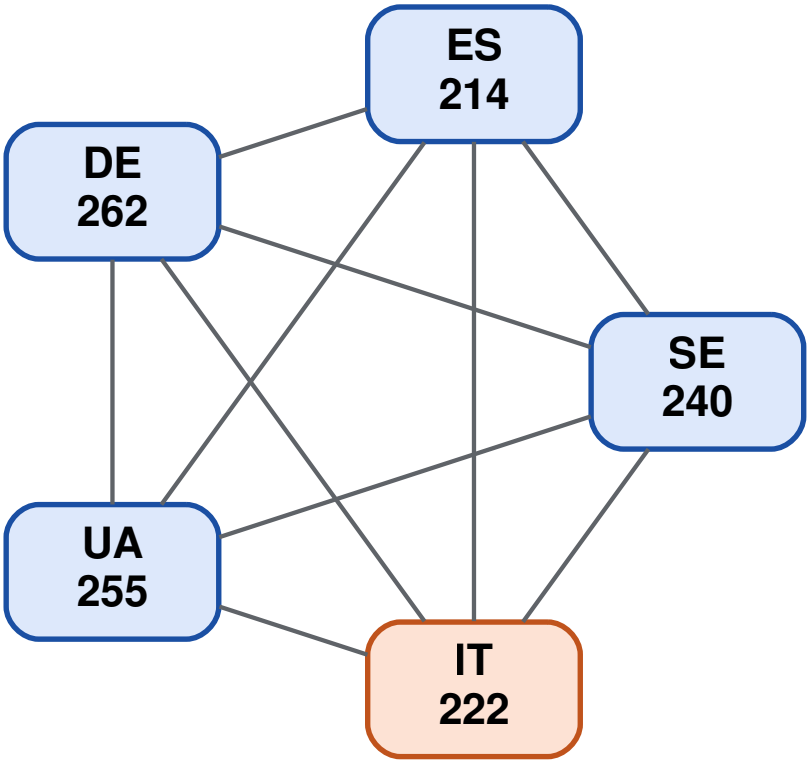


Diagram 1 — Three European country gateways in a full trunk mesh. Every pair has a bidirectional trunk over port :5302 with HMAC handshake.

Routing

TG `240xx` → SE · TG `262xx` → DE · TG `222xx` → IT.

Flow

An SE client keys TG `22299`. SE's prefix table maps `222 → IT`, so SE sends to IT. IT delivers locally and fans the audio out to DE (if it has shown interest in `22299` recently); SE is excluded as the source.

This layer is the foundation. Everything else stacks on top of it.

2. National trunk topology joined at the country gateway

Italy adds per-Zone reflectors that form a **national trunk mesh** together with IT 222. Foreign countries see only IT 222 — the Zone prefixes are not advertised internationally. The country gateway is the **hinge** between the international mesh and the national mesh.

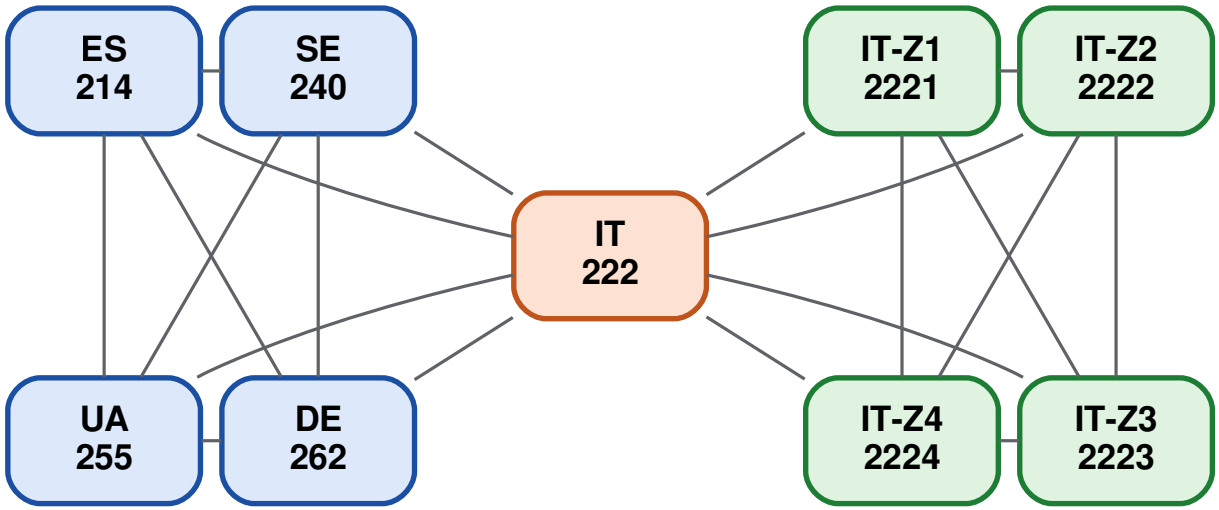


Diagram 2 — Two domains sharing a single node: the international trunk mesh on the left and the Italian national trunk mesh on the right, joined at IT 222. (Z5..Z9 omitted; same pattern.)

Italian Zone prefixes (BrandMeister)

Prefix	Reflector	Coverage
`2221`	IT-Z1	Nord-Ovest (Liguria, Piemonte, Val d'Aosta)
`2222`	IT-Z2	Lombardia
`2223`	IT-Z3	Triveneto (FVG, Trentino-AA, Veneto)
`2224`	IT-Z4	Emilia-Romagna
`2225`	IT-Z5	Toscana
`2226` – `2229`	IT-Z6..Z9	Centro, Puglia, Sud, Sicilia

Ownership vs. reachability

Each reflector's prefix table is built from *its own* config: local prefix plus the `REMOTE_PREFIX` of each `[TRUNK_x]` section. Prefixes are not gossiped between peers. That means SE only knows the prefixes it has trunks for — namely `222` (IT) and `262` (DE). SE has no idea that `2222` exists, even though Z2 owns it inside Italy.

TG	Owner (globally)	SE's view	IT 222's view
`22221`	IT-Z2 (longest match `2222`)	only `222` matches → send to IT 222	`2222` matches → Z2 is owner
`22299`	IT 222 (only `222` matches)	send to IT 222	own this TG
`2629`	DE (only `262` matches)	send to DE	send to DE

Flow

A **Z2 client** keying `22221` (Lombardia): Z2 owns `2222`, so it plays locally and fans out on the trunk side to interested peers in the national mesh — IT 222 and the other Zones. It does *not* trunk-forward to SE/DE (it has no trunks to them).

An **SE client** keying the same `22221`: SE's table has only `222` as a match, so it sends to IT 222 thinking IT is the owner. IT 222 receives it, sees that Z2 actually owns `2222` in its own trunk table, and **bridges the frame onward to Z2** via the gateway-routing path (`Reflector::shouldRelayInbound` returns true because IT has a prefix route for `2222`). Z2 plays locally and fans the audio out to its national peers (excluding IT as the source). The return leg works the same way: audio from a Z3 client reaches SE because IT's `isPeerInterestedTG` for `22221` was registered on the SE side — either by a previous PTT from SE, or proactively as soon as the Swedish operator selected (or monitored) `22221` on their SvxLink node, via `MsgPeerTgInterest` (type 129).

Reaching international TGs *from* a Zone — `ROUTABLE_PREFIXES`. The gateway routing above carries a *foreign* talker *into* a Zone. The opposite direction — a **Zone client calling an international TG** (say DE `2629`, or any TG owned out in the international mesh) — does **not** work by default: a Zone's prefix table holds only `222` and the other Zone prefixes, so `2629` matches nothing, has no owner, and stays local. To open that path, declare the foreign prefixes as routable via the gateway on the Zone's uplink to IT 222 — e.g. set `ROUTABLE_PREFIXES=262,240` (DE, SE, …) alongside `REMOTE_PREFIX=222` in the Zone's `[TRUNK_x]` section facing IT 222. Now `2629` matches the routable `262`, so the Zone forwards it to IT 222, which — in the international mesh with a `262` trunk to DE — relays it onward; the return audio comes back via peer interest.

A multi-trunk Zone (inside the national mesh) must name each foreign prefix explicitly — there is no automatic propagation. A region trunked *only* to IT 222 (a single-uplink leaf) can instead use `ROUTABLE_PREFIXES=*`, a default route reaching any TG via its one uplink; do **not** use `*` on a multi-trunk Zone (it logs a startup warning and is ambiguous about which leg to use).

> **CLUSTER TGS ARE FOR UNOWNED TGS, NOT CROSS-MESH ROUTING** Gateway prefix routing handles foreign-to-regional reach automatically as long as IT 222 has a `[TRUNK_x]` section pointing at the regional reflector — regional TGs do not need to be listed in `CLUSTER_TGS`. `CLUSTER_TGS` is the right tool for TGs that **cannot** be resolved by prefix because no reflector owns them: TG `91` for worldwide chat (international domain), or `112` for nationwide broadcast inside a country (national domain). Configure those on every reflector that should join the broadcast.

Variant: sub-trunk star

The national side does not have to be a full mesh. IT 222 can delegate sub-prefixes of its own `222` over ordinary trunks to Zones that each trunk *only* to IT 222. The Zones still **own** their prefixes — that is the whole difference from a satellite, which owns nothing and inherits its parent's identity (§3). A Zone may own more than one prefix: `REMOTE_PREFIX` is a comma-separated list, so one reflector can serve Lombardia and Emilia-Romagna together.

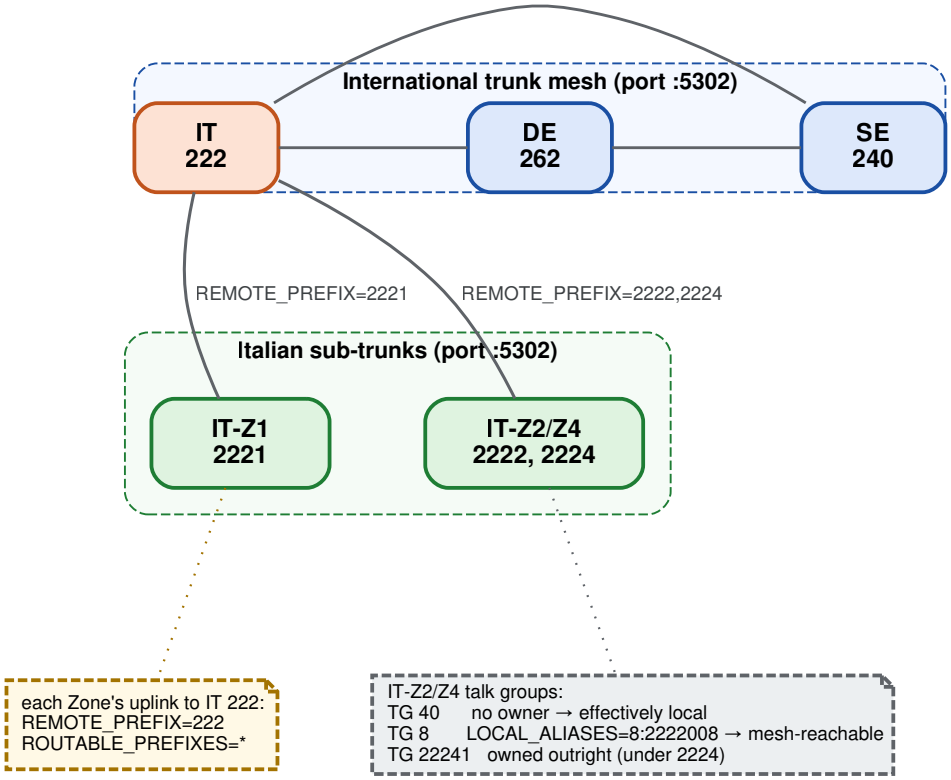


Diagram 3 — IT 222 owns `222` and delegates `2221` and `2222,2224` to Zone reflectors over ordinary trunks. A star, not a full mesh: each Zone holds exactly one uplink, so its uplink section carries `ROUTABLE_PREFIXES=*`.

Each Zone has exactly one trunk, so its uplink takes the single-uplink-leaf `ROUTABLE_PREFIXES=*` described above rather than an explicit prefix list.

Delegation, not collision

IT 222 owns `222`; Z1 owns `2221`. TG `22213` matches both, and longest-prefix-match hands it to Z1. That overlap is the mechanism, not a hazard.

TG	Longest match	Owner
`22213`	`2221`	IT-Z1
`22241`	`2224`	IT-Z2/Z4
`22299`	`222`	IT 222
`2629`	none locally → `*` route	up to IT 222, then DE

A **Z1 client keying** `2629` (DE) reaches Germany because `2629` matches nothing Z1 owns and the uplink's `*` makes IT 222 the default route: Z1 → IT 222 → DE. Inbound, an SE client keying `22241` lands on IT 222, whose gateway prefix routing bridges it down to IT-Z2/Z4 exactly as in the full-mesh case.

A Zone mixes local-only and mesh-reachable TGs

Owning `2222,2224` does not mean every TG on that reflector is routable. Three kinds coexist.

TG on IT-Z2/Z4	Config	Reach
`22241`	owned outright (under `2224`)	routed across the mesh by prefix
`40`	nothing — matches no prefix	**effectively local**: no reflector owns it
`8`	`LOCAL_ALIASES=8:2222008`	dialed as `8` locally, `2222008` on the mesh

`40` stays a private Zone talk group and `8` is the club TG operators keep dialing by its short number while the mesh routes it as `2222008` under the Zone's own `2222` prefix. Mind the cost of the `*` route, though: it matches everything no longer prefix claims, so `40` is still forwarded up the uplink, and IT 222 discards it only on arrival — it matches neither IT's `LOCAL_PREFIX` nor the Zone's `REMOTE_PREFIX`. Nobody hears it, but it crosses the wire. With an explicit `ROUTABLE_PREFIXES=262,240` it would never egress at all.

Full mesh or star?

Property	Full mesh	Sub-trunk star
TCP connections per Zone	M – 1 within the national mesh (Zones + IT 222)	1 (to IT 222)
NAT story	Zones need inbound reachability from other Zones + IT 222	Zone dials out; only IT 222 needs an inbound port
IT 222 outage	Zones keep talking to each other; international reach is lost	Zones are isolated from each other and from abroad
Zone-to-Zone audio	direct, one hop	two hops, relayed by IT 222
Foreign → regional TG reach	gateway prefix routing on IT 222 (`shouldRelayInbound`)	same
Reaching international TGs from a Zone	list foreign prefixes in `ROUTABLE_PREFIXES` per leg	`ROUTABLE_PREFIXES=*` on the one uplink
Prefix advertised upstream	only `222` (Zones not visible to SE/DE)	only `222`

Either way, foreign reflectors carry no Italian internal prefix table — the country presents one face upstream (`222`) — and IT 222 is a single point of failure for international reach into or out of Italy, unless it is run as a twin pair (§4).

3. Satellite under a regional reflector

Hybrid layout: keep the §2 two-domain structure (intl mesh + national mesh joined at IT 222), and let one Zone host its own satellite reflector. The Zone itself is *not* a satellite — it wears two hats: trunk peer in the national mesh, parent to its own satellite. A satellite is a **leaf**: it owns no prefix, accepts no children of its own, and inherits the parent's identity. That is why a Zone (which owns `2222`) is sub-trunked in §2 rather than made a satellite. Two motivations apply, often together:

- **NAT traversal.** The downstream site (e.g. a Milano club, MI) only needs one outbound TCP connection to Z2. No port-forwarding, no public IP — useful when the sub-group's site sits behind carrier-grade NAT or a firewall the Zone admin doesn't control.
- **Administrative delegation.** The Zone admin (e.g. of IT-Z2 / Lombardia) lets a sub-group bring up their own reflector with their own user accounts and passwords, *without* sharing Z2's `[USERS]` / `[PASSWORDS]` database. Z2 admin holds Z2's user DB and the `[SATELLITE] SECRET` that MI presents to attach; MI admin holds MI's own `[USERS]` / `[PASSWORDS]` — only MI's V2 clients authenticate against it. Audio federates over the satellite link while authentication state stays separated on each side.

> **TRUST MODEL** Pin each satellite to its own credential with a `SECRET_<id>` entry in `[SATELLITE]`, matched against the satellite's `SATELLITE_ID`. Per-id wins, and an HMAC mismatch on a pinned id is rejected with no fallback to the shared `SECRET` — so a leaked secret from one club cannot impersonate another. Use it whenever more than one operator runs satellites under the same parent. A single shared `SECRET` is fine for a one-operator deployment.

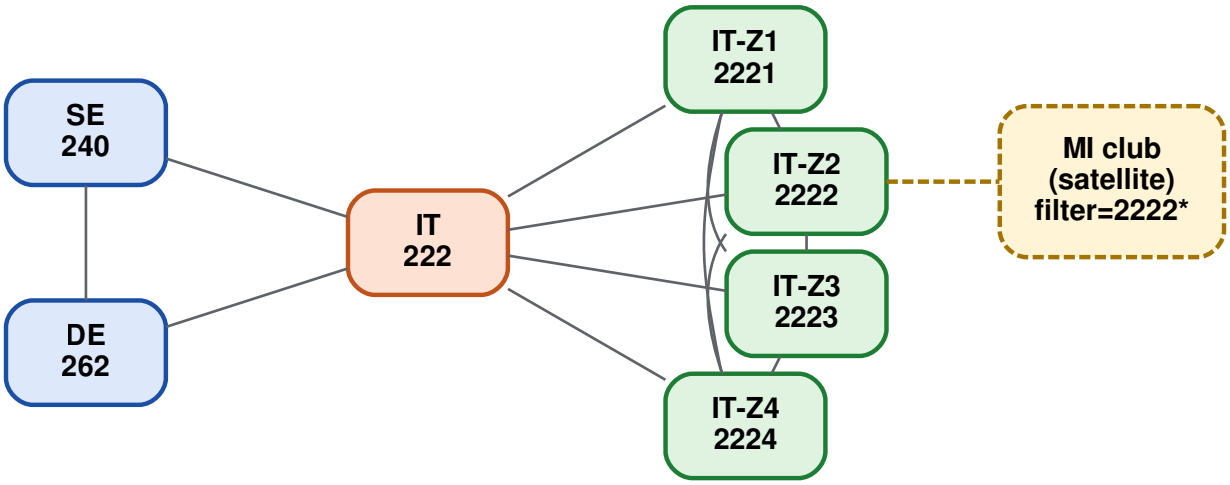


Diagram 4 — International + national trunk mesh joined at IT 222, plus a satellite reflector (MI club, NAT'd) hanging off Z2 over port :5303. Z2 simultaneously serves as national trunk peer and satellite parent.

Flow

- **MI client keys `22221`:** MI mirrors up to Z2 (Lombardia). Z2 owns `2222` — plays locally, fans on the trunk side to interested peers in the *national* mesh (IT 222 / Z1 / Z3 / Z4 if they've shown

interest), and skips MI on the satellite side because MI is the source. Z2 does not trunk-forward to SE/DE — it has no trunks to them.

- **SE client keys `22221`:** SE only knows `222 → IT`. SE→IT 222 over the trunk. IT 222 isn't the owner of `2222`, but its gateway-routing rule (`shouldRelayInbound` finds a `2222` route in IT's trunk table) bridges the frame to Z2. Z2 plays locally and mirrors down to MI through the satellite link (filter `2222*` matches). The return leg works symmetrically through `isPeerInterestedTG`.
- **MI client keys `22241`** (Emilia-Romagna, owned by Z4): MI→Z2 (satellite up). Z2 isn't the owner — its trunk-side forwarding sees `2224` and trunk-sends to Z4. Z4 plays locally and fans to interested peers, including Z2 *only if* Z2 has previously PTT'd on `22241`. Consequence: the first MI→`22241` PTT works outbound but won't get return audio until Z2 is registered as interested at Z4. The MI→Z2→Z4 PTT itself registers Z2's interest, so for the next ~10 min audio from any other client on `22241` flows Z4→Z2→MI.

> **FILTER CAVEAT** A restrictive `SATELLITE_FILTER` like `2222*` prevents MI from reaching anything outside Lombardia: filter is bidirectional, so MI suppresses non-`2222*` outbound and Z2 drops non-`2222*` forwards going down. The MI→`22241` example above only works if the filter is broadened to `222*` (or unset). Use a tight filter only when the satellite truly is single-purpose (e.g. a regional repeater that should never hear other Zones' traffic). Mind the syntax: entries are **exact** unless they end in `*` — `2222` matches only literal TG `2222`, while `2222*` matches everything under Lombardia. The shared TG-filter syntax is exact `26200`, prefix `24*`, range `2427-2438`, comma-separated.

4. HA twin pair

A **twin** is a purpose-built link type for running two reflector instances as a redundant pair. Both nodes declare the *same* `LOCAL_PREFIX`, mirror their full `TGHandler` state to each other over a `[TWIN]` link, and present as a single logical reflector to the rest of the world. SvxLink nodes list both endpoints in their `HOSTS` setting for transparent failover.

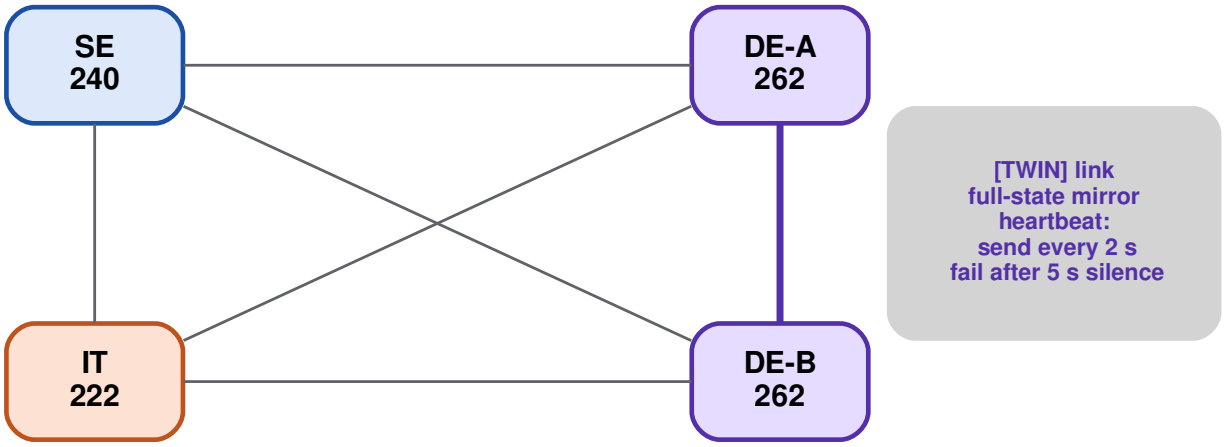


Diagram 5 — DE is operated as a twin pair (DE-A and DE-B both own `LOCAL_PREFIX=262`). External peers (IT, SE) hold one `[TRUNK_x]` section with `PAIRED=1` and `HOST=de-a,de-b`; each frame goes to one socket at a time, sticky with failover. The bold purple `[TWIN]` link mirrors full state symmetrically.

What's different from trunk and satellite

Property	Trunk	Satellite	Twin
Directionality	symmetric	one-way bias (parent→satellite)	symmetric
Prefix ownership	distinct per node	n/a	**identical** on both nodes
Filtering	`isSharedTG` / `isOwnedTG`	none (all TGs)	none (full mirror inside the pair)
Peers per section	1	many satellites per parent	**exactly 1** (strictly pairwise)
External trunk ownership	each node holds its own	parent-only	both hold; external peer picks one socket per frame
Heartbeat cadence (TX = send interval, RX = silence timeout)	10 s / 15 s	10 s / 15 s	**2 s** / **5 s** (LAN-close)

Flow

- **SvxLink client connects to DE-B** and PTTs on a German TG. DE-B handles the audio locally (it owns `262`) and mirrors state to DE-A over the twin link so DE-A's clients can't grab the same TG simultaneously. External trunks: DE-B forwards on its own external trunks to interested peers; DE-A does *not* re-forward (otherwise external peers would see duplicates).
- **An IT client keys a German TG**: IT's `[TRUNK_DE]` section has `PAIRED=1` and `HOST=de-a,de-b`, so IT holds two TCP sockets but sends each frame to its sticky-selected endpoint (say DE-A). DE-A delivers locally, mirrors to DE-B over the twin link so DE-B's local clients hear it too, and DE-B suppresses re-forwarding back out.
- **DE-A goes down**: IT's TCP socket to DE-A drops; IT's next frame fails over to DE-B with no holdoff. DE-B keeps serving; its clients are unaffected. When DE-A recovers, the twin handshake resyncs `TGHandler` state.
- **Twin link itself goes down (split-brain)**: both twins keep operating fully and independently. Cross-pair talker arbitration is lost for the duration — duplicate-talker artifacts may appear if clients on both nodes PTT the same TG simultaneously. External trunks remain healthy in both directions.

> **CROSS-TWIN ARBITRATION** When both twins' clients PTT the same TG at the same instant, the priority-nonce in `MsgPeerHello` tie-breaks (lower wins, loser yields). This is the same mechanism trunks use for cross-site arbitration — reused intra-pair. Two new message types (`MsgTwinExtTalkerStart` / `Stop`, types 123/124) carry external-trunk talker state across the twin so the partner won't let local clients key a TG that's already busy on an external peer.

When to use it

Twin layout is the right answer when you need **HA failover for clients** on a single reflector site — for instance, two physical servers on the same LAN, both with public IPs, fronting a cluster of SvxLink nodes. It composes with everything else: the twin pair can simultaneously be a country gateway in the international mesh (§1), a member of a national trunk topology (§2), or the parent of a club satellite (§3). Don't reach for twins just to share traffic across two boxes — that's what plain trunks are for.

Overlay: local aliases

Not a topology shape but an overlay that composes with all five: a way to make an existing **short local TG** reachable from the mesh without renumbering it on the clients. Routing follows the number, so a TG matching no configured prefix has no owner and stays **local-only** — yet a club or legacy talk group like `8` is exactly the kind of number operators have in their muscle memory and their node configs. `LOCAL_ALIASES` (in `[GLOBAL]`) resolves the tension by **re-homing** the short TG under one of the reflector's own prefixes, giving it a globally-unique number while local users keep dialing the short one — e.g. `LOCAL_ALIASES=8:2220008` alongside `LOCAL_PREFIX=222` on IT 222.

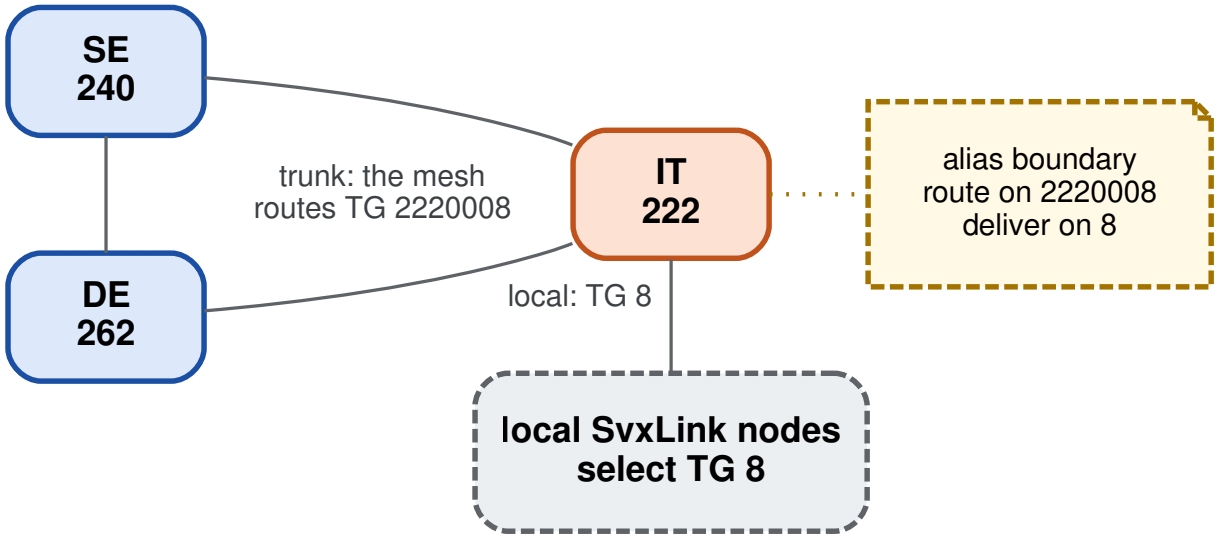


Diagram 6 — IT 222 re-homes its local TG `8` as `2220008`. Foreign peers route the long number by ordinary longest-prefix-match; IT translates at the trunk boundary and its own nodes keep selecting `8`.

Syntax and direction

Comma-separated entries of the form `localTG:aliasTG[/dir]`, where `dir` is `in` or `out`; omitted means two-way.

- **two-way** (`8:2220008`): local talk on `8` is exported to the mesh as `2220008`, and mesh traffic on `2220008` is delivered to local clients as `8`.
- **`/out`** (`90:2220090/out`): publish-only — local talk is exported, inbound mesh traffic is not folded back onto the short number.
- **`/in`** (`91:2220091/in`): listen-only — mesh traffic is delivered locally, the short number is never exported.

Ownership rule

`aliasTG` must fall under one of *this* reflector's `LOCAL_PREFIX` values, checked with the same longest-prefix match used for TG ownership everywhere else. An entry whose `aliasTG` is not owned is rejected at config load — logged as an error and skipped, while the reflector still starts and the

remaining valid entries take effect. A reflector can therefore never alias its way onto a number another peer owns: you may only re-home a TG into space you already own.

Routing on the long number, delivery on the short one

The alias is a translation applied at the **trunk boundary**, not a rename. Prefix routing, owner-relay fanout and peer-interest bookkeeping all run on the long number, because that is the identity the rest of the mesh matches prefixes against. Local delivery and talker arbitration run on the short number, because that is what local clients selected. Everything on the local side of the boundary — clients and satellites alike — sees `8`; everything across a trunk sees `2220008`. A client that dials the long number directly is folded onto the short one the moment it selects or monitors the TG (*rewrite-on-select*), so the two are always one conversation locally.

Flow

- An **SE client keys** `2220008`: SE's prefix table matches `222 → IT`, so the audio crosses the trunk to IT exactly like any other owned TG. IT translates at the boundary and delivers it to its local nodes on TG `8`.
- An **IT client keys** `8`: IT exports it to the mesh as `2220008`, reaching peers that have registered interest in that TG. Locally it is an ordinary TG `8` transmission.

Because `aliasTG` must be owned locally, each reflector aliases into its *own* prefix space: in §2 an Italian Zone re-homes its club TGs under its Zone prefix (`2222…` for Z2) while IT 222 re-homes under `222…`. In a twin pair (§4), give **both** twins identical `LOCAL_ALIASES` — each twin independently applies the translation on its own trunk sections.

> **ALIASES ARE NOT CLUSTER TGS** `CLUSTER_TGS` broadcasts an *unowned* TG to every peer regardless of prefix. `LOCAL_ALIASES` does the opposite: it pulls an unowned short number *into* your owned prefix space so that ordinary longest-prefix-match routing takes over. Reach for an alias when the TG belongs to you; reach for a cluster TG when it belongs to everyone.

Picking a topology

Constraint	Use
Just one reflector per country, no regional split needed	International only (§1)
Each Zone has a public IP; want regional autonomy and resilience to gateway outage	National trunk topology, full mesh (§2)
Zones own their prefixes but sit behind NAT, or you want one simple uplink per Zone instead of a full mesh	National trunk topology, sub-trunk star (§2)
§2 layout overall, plus a sub-group needs its own reflector — either NAT'd or with its own user DB to delegate (`[USERS]` / `[PASSWORDS]` separated)	Satellite under a regional reflector (§3)
HA failover for SvxLink clients on a single site (two boxes, same LAN)	Twin pair (§4) — composes with §1–§3
An existing short local TG (e.g. `8`) must stay dialable locally but become reachable across the mesh	`LOCAL_ALIASES` overlay — composes with §1–§4

The shapes compose: the international layer (§1) is always present, and §2 and §3 are ways to fan out *inside* a country. §4 is orthogonal — any single reflector role above can be played by a twin pair instead, and the `LOCAL_ALIASES` overlay applies to any of them.

Appendix: MCC ranges and cluster-TG numbering

Talk-group prefixes align with the ITU-T E.212 Mobile Country Code (MCC) plan, so prefix ownership is self-documenting and globally consistent.

Range	Use
`0`	**Special.** TG `0` is a protocol sentinel meaning "no TG selected." V2 clients authenticate with `tg=0` and then select a real TG with `MsgSelectTG`. Audio is never sent or received on TG `0`.
`001` , other `0xx`	ITU test networks. Not assigned to any country.
`1xx`	Reserved by ITU. Not assigned to any country.
`2xx`	**Europe.** Lowest country MCC: `202` Greece. (`200` , `201` are unassigned.)
`3xx`	North America (incl. Greenland, Caribbean)
`4xx`	Asia, Middle East
`5xx`	Oceania
`6xx`	Africa
`7xx`	South & Central America
`8xx` , `9xx`	Reserved / non-country use (rare assignments)

Safe cluster-TG ranges: `100-199` and `800-999`. Pick cluster TGs (the kind that bypass prefix ownership — see §2) from these ranges:

- The `1xx` block is reserved by ITU and contains no country MCC.
- The `8xx` and `9xx` blocks are reserved / non-country (rare assignments only).

A cluster TG drawn from either range cannot collide with any current or future country prefix (countries live entirely in `200`–`799`). Combined that's roughly 300 available numbers — more than enough for worldwide, nationwide, and any thematic broadcast TGs you want to define.

Avoid `200`–`799` (country prefix collisions) and the `0xx` / `1`–`99` ranges (the latter are technically safe but unconventional for DMR/SvxLink TG numbering — short numbers are rare in practice). The well-known TG `91` (worldwide chat) is a pre-existing convention that predates this recommendation; new cluster-TG assignments should follow the `100-199` / `800-999` rule.

Properties that hold across all variants

- Longest-prefix-match decides the owner of any given TG.
- Routing follows the **number, not its length**: a TG goes to the owner of its longest matching prefix that exists in the mesh (e.g. `22210` → `222`, `26201` → `262`, when those country prefixes are owned). A TG matching **no** configured prefix has no owner and stays **local-only** — it relays among the originating reflector's own nodes but is never trunked — unless a link carries `ROUTABLE_PREFIXES=*`, whose default route forwards it to the uplink, where the peer drops it as unowned. Local TG numbering is the operator's choice; only numbers under the agreed prefix scheme are reachable across the mesh. A local-only number can be given mesh reach without renumbering the clients, via the `LOCAL_ALIASES` overlay.
- Aliases translate at the trunk boundary only: the mesh routes the long owned number, local clients and satellites see the short one.
- Trunk relay is single-hop. Satellite mirror is independent of that rule.
- Owner fanout is gated by **peer interest** with a 10-minute idle window. Interest is registered when a peer first transmits on a TG, or proactively when it advertises a selected / monitored TG via `MsgPeerTgInterest`.
- Source-side exclusion in the owner's fanout prevents echo loops.
- `SATELLITE_FILTER` is bidirectional and advertised to the parent.
- Talker arbitration uses the random 32-bit nonce exchanged in `MsgPeerHello` — lower wins. The loser defers and blocks local clients from interrupting.